



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

CSA0619– Design and Analysis of Algorithms

**Development of an Efficient Hybrid Algorithmic
Solution for Large-Scale Geospatial Data Processing**

Assignment Report

Submitted by

P. C. Vamsidhar Reddy (192465045)

M. Maruthi (192425257)

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.E./B.Tech. Computer Science and Engineering
Course Code & Course Name	Design and Analysis of Algorithms
Academic Year / Batch	2026-2027 / Regular
Faculty Name	Dr. F. Mary Harin Fernandez
Assignment Title	Development of an Efficient Hybrid Algorithmic Solution for Large-Scale Geospatial Data Processing
Date of Issue	31 Aug 2026
Date of Submission	1 September 2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication
Bloom's Taxonomy Level	L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient processing of large-scale real-world data with reduced computational time and resource utilization

B. Assignment Problem / Challenge

Modern infrastructure-planning applications process large volumes of **geospatial coordinate data**, making computationally efficient algorithms essential for identifying spatial relationships among facilities. Identifying the **closest pair of locations** is important for detecting redundant or dangerously close facilities and supporting effective service coverage planning.

Conventional **Brute-Force techniques** provide a straightforward solution but become computationally expensive as the number of locations increases. **Divide-and-Conquer techniques**

improve scalability by recursively partitioning the dataset and reducing unnecessary distance computations.

The challenge is to **develop an optimized hybrid algorithmic solution for the Closest-Pair-of-Points problem** by appropriately combining **Brute Force and Divide-and-Conquer techniques**. The developed solution must determine an effective switching threshold between the two approaches and be supported by **theoretical complexity analysis and experimental evaluation using a real-world geospatial dataset**.

C. Problem Statement

An infrastructure-planning agency maintains a large, publicly available geospatial dataset of facility locations—for example, the OpenFlights airports dataset or a cell-tower/weather-station dataset containing coordinate records. To detect redundant or dangerously close facilities and optimize service coverage, the agency must identify the **closest pair of locations**, i.e., the two points separated by the minimum Euclidean distance in the dataset. Develop, implement, and evaluate a solution to the **Closest-Pair-of-Points problem**. Implement **(i) a Brute-Force algorithm** that evaluates all pairs with complexity and **(ii) a Divide-and-Conquer algorithm** that recursively partitions the point set about the median x-coordinate and evaluates points across the dividing strip with complexity. Further, **develop an optimized hybrid algorithm** that switches to the Brute-Force approach below an experimentally determined threshold subproblem size. Analyze the theoretical efficiency of each approach using **Big-O, Ω , and Θ notations**, and solve the recurrence using the **Master Theorem**. Experimentally evaluate all three algorithms on the selected dataset using increasing input sizes, **for example, , , , and higher input sizes wherever computationally feasible**, measuring **execution time, number of distance computations/comparisons, memory utilization, and scalability**. Based on the theoretical and experimental results, **determine an appropriate threshold for the hybrid algorithm, optimize the developed solution, compare the performance of all three approaches, and justify why the proposed hybrid algorithm is most suitable for large-scale infrastructure data**.

Expected Development

The work should **not merely implement the existing Brute-Force and Divide-and-Conquer algorithms**. The solution should demonstrate algorithmic development through:

- **Hybridization of Brute Force and Divide-and-Conquer techniques** for the Closest-Pair-of-Points problem.
- **Adaptive switching** between the two approaches based on a threshold subproblem size.
- **Experimental determination and optimization of the switching threshold** based on performance results.
- **Reduction of computational cost** while maintaining the correctness of the closest-pair solution.
- **Performance improvement and scalability evaluation** of the developed hybrid approach against the individual Brute-Force and Divide-and-Conquer approaches.

D. Requirements and Constraints

The following requirements and constraints shall be considered:

- Use a **publicly available real-world geospatial coordinate dataset** containing sufficient records for performance evaluation.
- Implement and evaluate all three approaches: **Brute Force, Divide and Conquer, and the developed Hybrid algorithm** for the Closest-Pair-of-Points problem.

- Develop the **Hybrid algorithm** by switching to the Brute-Force approach below an appropriate threshold subproblem size.
- Determine and justify the **switching threshold experimentally** based on performance results.
- Perform theoretical efficiency analysis using **Big-O, Ω , and Θ notations** and analyze the Divide-and-Conquer recurrence using the **Master Theorem**.
- Evaluate the algorithms using increasing input sizes, for example, , , , and **higher input sizes wherever computationally feasible**.
- Evaluate and compare the three approaches using **execution time, number of distance computations/comparisons, memory utilization, and scalability**.
- Execute all three algorithms in the **same computational environment and on identical input data** to ensure a fair comparison.
- Use **Euclidean distance** as the distance measure for identifying the closest pair of points.
- Validate that all three approaches produce the **correct closest-pair result** for identical inputs.
- Higher input sizes shall be considered only where computationally feasible, taking into account the **computational cost of the Brute-Force approach**.
- Clearly document the **dataset source, experimental environment, threshold value, assumptions, limitations, and implementation details**.

E. Student Work

1. Problem Understanding and Formulation

- The Closest-Pair-of-Points problem asks for the two geospatial locations with the minimum Euclidean separation. "
"In infrastructure planning, the result can be used as an indicator of potentially redundant or dangerously close facilities, "
"and can support service-coverage planning. The assignment requires Brute Force, Divide and Conquer, and a developed Hybrid algorithm, "
"with an experimentally selected switching threshold.
- Dataset. The OpenFlights Airports Database was selected because it is publicly available and contains airport/facility coordinates. The source describes fields including Airport ID, Name, City, Country, IATA, ICAO, Latitude and Longitude. The working extract supplied with this submission contains 100 valid airport records from the OpenFlights snapshot.
- Working dataset attributes: Airport ID, Name, Latitude, Longitude. Invalid/missing coordinate records are excluded. The experimental CSV is `openflights_working_extract_100.csv`.
- Expected input: a collection of points represented as (latitude, longitude, name, identifier). Expected output: the two point records having minimum Euclidean distance, together with distance-computation count, execution time and peak memory measurement.
- Euclidean distance is used exactly as required by the assignment: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. The coordinates are treated as a 2-D numerical plane; the experiment therefore does not claim a geodesic/earth-surface distance.
- Computational challenge. Brute Force evaluates $n(n-1)/2$ pairs, so its work grows quadratically. As n increases, distance evaluations and execution time rise rapidly. Divide and Conquer reduces the number of candidate comparisons by spatial partitioning. The

Hybrid method further reduces recursive overhead by solving small subproblems directly with Brute Force.

- Assumptions and constraints: identical input data and environment are used for all algorithms; Euclidean distance is mandatory; the algorithms must return the same closest pair; the threshold must be justified experimentally; and higher input sizes are limited by the cost of the Brute-Force baseline.

2. Application of Course Knowledge

CO1 – Algorithm Analysis

The work applies algorithmic problem-solving, efficiency analysis, Big-O, Ω and Θ notation, recursive/non-recursive analysis, and the Master Theorem.

CO2 – Brute Force

Every unordered pair is evaluated. For n points this produces $n(n-1)/2$ distance computations, giving $\Theta(n^2)$ time.

CO3 – Divide and Conquer

Points are recursively divided around the median x-coordinate

The closest pair is found in each half, then a vertical strip around the division is examined using the y-order to avoid testing all cross-half pairs.

Contribution to the Hybrid algorithm

CO1 provides the complexity framework and threshold rationale; CO2 supplies the small-subproblem solver; CO3 supplies the scalable recursive structure. The Hybrid algorithm combines these by using Brute Force when $n \leq T$ and Divide and Conquer otherwise.

3. Solution / Design / Methodology

Methodology followed: Input Dataset → Data Preparation → Brute-Force Implementation → Divide-and-Conquer Implementation → Hybrid Algorithm Development → Threshold Determination → Complexity Analysis → Experimental Evaluation → Optimization → Performance Comparison → Final Decision.

3.1 Brute-Force Pseudocode

```
BRUTE-FORCE(points)
best ← ∞
for i ← 0 to n-1:
    for j ← i+1 to n-1:
        d ← EuclideanDistance(points[i], points[j])
        if d < best:
            best ← d
            bestPair ← (points[i], points[j])
return bestPair, best
```

3.2 Divide-and-Conquer Pseudocode

DIVIDE-AND-CONQUER(points sorted by x and y)
 if $n \leq 3$:
 return BRUTE-FORCE(points)
 split at median x-coordinate
 leftResult $\leftarrow$ recursive left half
 rightResult $\leftarrow$ recursive right half
 $\delta \leftarrow$ smaller of leftResult and rightResult
 strip $\leftarrow$ points within δ of the division line, ordered by y
 check only the bounded number of following y-neighbours in strip
 return the smallest result

3.3 Developed Hybrid Pseudocode

HYBRID(points, threshold T)
 sort points by x and y
 HYBRID-REC(px, py, T)
 if $|px| \leq T$:
 return BRUTE-FORCE(px)
 split at median x
 leftResult $\leftarrow$ HYBRID-REC(left_x, left_y, T)
 rightResult $\leftarrow$ HYBRID-REC(right_x, right_y, T)
 $\delta \leftarrow \min(\text{leftResult}, \text{rightResult})$
 build y-ordered dividing strip
 compare the bounded y-neighbours in the strip
 return minimum result

3.4 Flowchart / Design

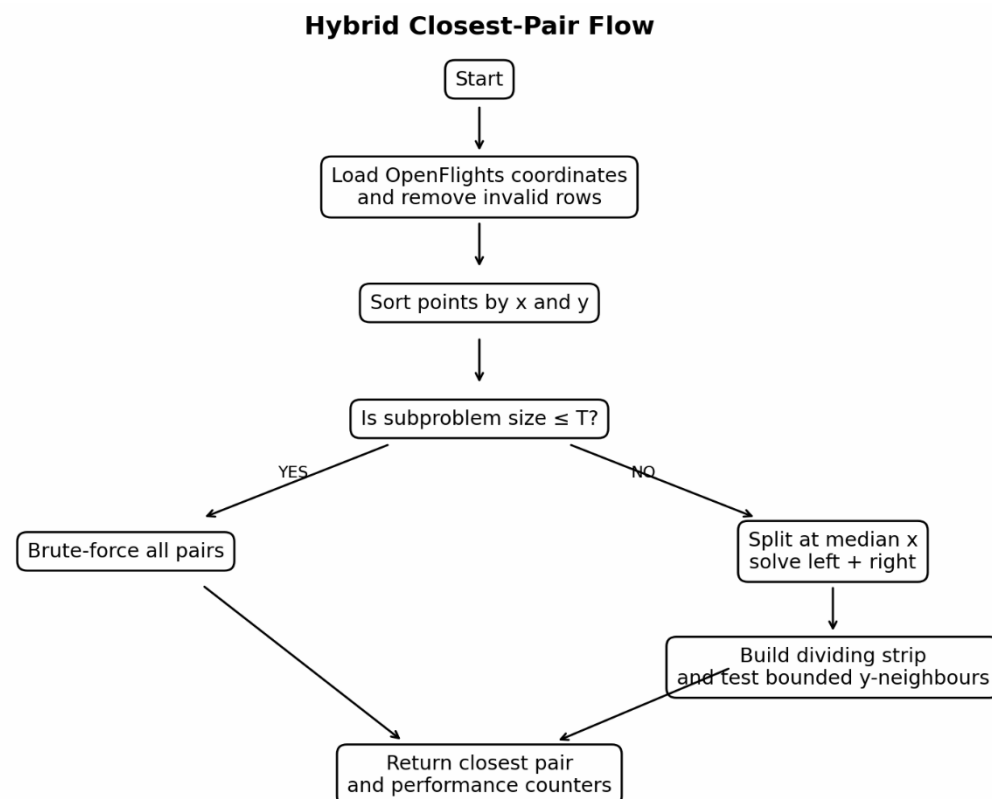


Figure 1. Flow of the proposed Hybrid Closest-Pair algorithm.

3.5 Hybrid Switching Strategy

The switching rule is $n \leq T \rightarrow$ Brute Force; $n > T \rightarrow$ Divide and Conquer. This is an adaptive implementation of the standard hybrid-algorithm principle: small subproblems are handled by the simpler method, while larger subproblems use the asymptotically efficient recursive method.

3.6 Method Used to Determine the Threshold

Threshold candidates $T \in \{4, 8, 12, 16, 20, 24, 32\}$ were tested on the same OpenFlights working extract. For input sizes 10, 20, 40, 60, 80 and 100, each configuration was executed five times and the median execution time was recorded. The threshold with the lowest mean Hybrid execution time across the tested sizes was selected.

Threshold	Mean Hybrid Time (s)	Relative to Best
16	0.000237	1.000×
12	0.000246	1.038×
20	0.000260	1.099×
24	0.000266	1.124×
8	0.000279	1.178×
32	0.000323	1.366×
4	0.000421	1.777×

Selected threshold: $T = 16$. It produced the lowest mean measured Hybrid execution time (approximately 0.000237 s) in this experiment. The result is an empirical threshold for this implementation and environment, not a universal constant.

3.7 Mathematical Complexity Analysis

Brute Force: $T(n) = n(n-1)/2$ distance evaluations. Therefore $O(n^2)$, $\Omega(n^2)$, $\Theta(n^2)$. The extra algorithmic space is $O(1)$, excluding the input storage.

Divide and Conquer: $T(n) = 2T(n/2) + O(n)$, because two halves are solved recursively and the strip is processed in linear time. By the Master Theorem, $a=2$, $b=2$, $f(n)=O(n)$, and $n^{\log_b a}=n$. This is Case 2, giving $\Theta(n \log n)$. Thus $O(n \log n)$ and $\Omega(n \log n)$ for the standard implementation. Auxiliary storage is $O(n)$.

Hybrid: for $n \leq T$, the base case costs $\Theta(n^2)$; for $n > T$, the recurrence is $2T(n/2)+O(n)$. With fixed constant T , the asymptotic worst-case complexity remains $\Theta(n \log n)$, while the constant factors can improve because small recursive subproblems avoid recursive overhead.

3.8 Design and Optimization Decisions

The implementation pre-sorts by x and y once at the top level and passes these ordered lists

through recursion. This avoids re-sorting every recursive subproblem. The strip is processed in y-order and only a bounded set of following neighbours is examined. The Hybrid base case uses direct pairwise evaluation, reducing recursion and bookkeeping overhead for small n . A distance-computation counter is included so that computational work can be compared independently of machine timing noise.

4. Use of Modern Tools

Python 3 was used as the implementation language. The experiment can be run from a command prompt or IDE. CSV processing uses Python's csv module and pandas is used for result analysis. `time.perf_counter()` measures execution time, `tracemalloc` measures peak Python memory allocated during each run, and `matplotlib` generates the performance graphs. The source code is provided separately as `hybrid_closest_pair.py` and the working dataset as `openflights_working_extract_100.csv`.

Execution command: `python hybrid_closest_pair.py --csv openflights_working_extract_100.csv -out results.csv --threshold 16`

Experimental environment: the code was executed in the supplied Python execution environment; all three algorithms were run in the same environment and on identical input data.

5. Results and Validation

All three implementations returned the same closest pair for every tested input size. For the 100-record working extract, the closest pair was Ottawa / Gatineau Airport and Ottawa Macdonald-Cartier International Airport, with Euclidean distance 0.22546189 coordinate units.

Chosen Threshold $T=16$: Benchmark Results

n	Brute s	D&C s	Hybrid s	Brute comps	D&C comps	Hybrid comps	Correct
10	2.8e-05	6e-05	2.7e-05	45	11	45	True
20	7.7e-05	0.00018	7.5e-05	190	26	91	True
40	0.00038	0.000335	0.000162	780	43	181	True
60	0.001006	0.00068	0.000293	1770	69	423	True
80	0.001739	0.000827	0.000377	3160	101	364	True
100	0.002587	0.00104	0.000487	4950	142	580	True

Figure 2. Benchmark results for the selected threshold $T=16$.

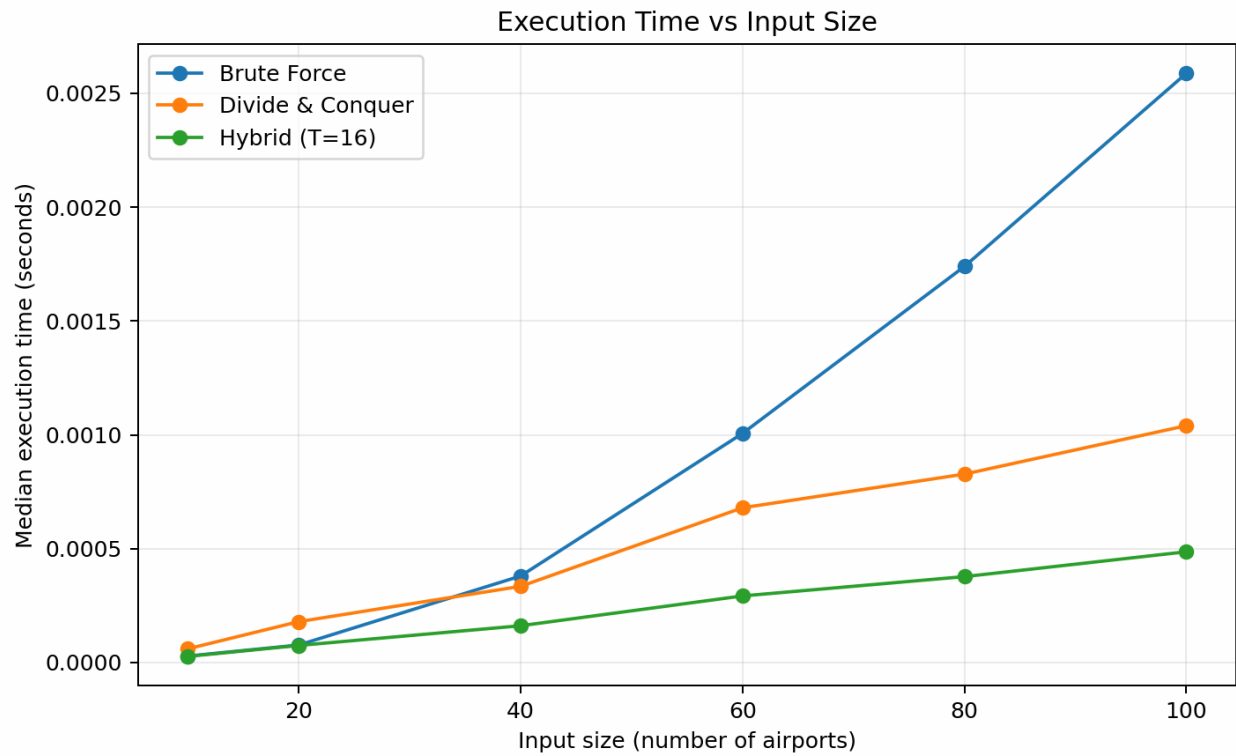


Figure 3. Execution time versus input size.

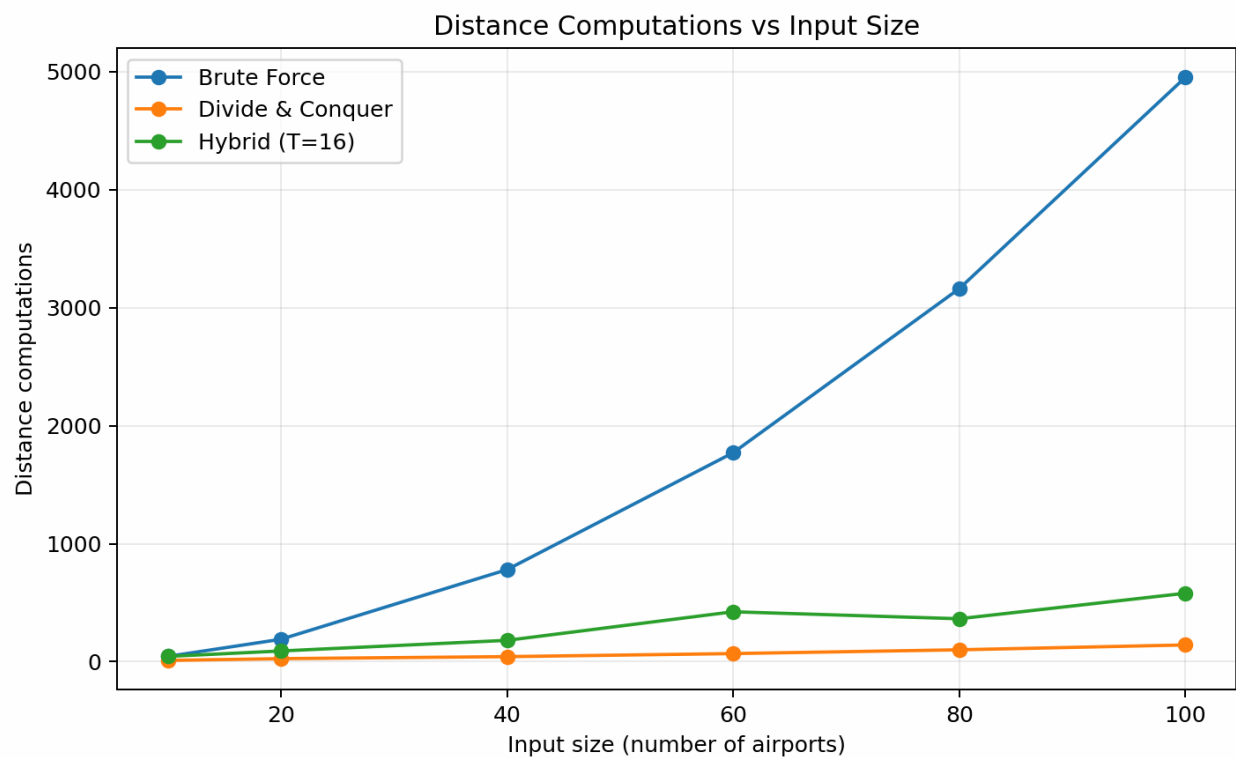


Figure 4. Distance computations versus input size.

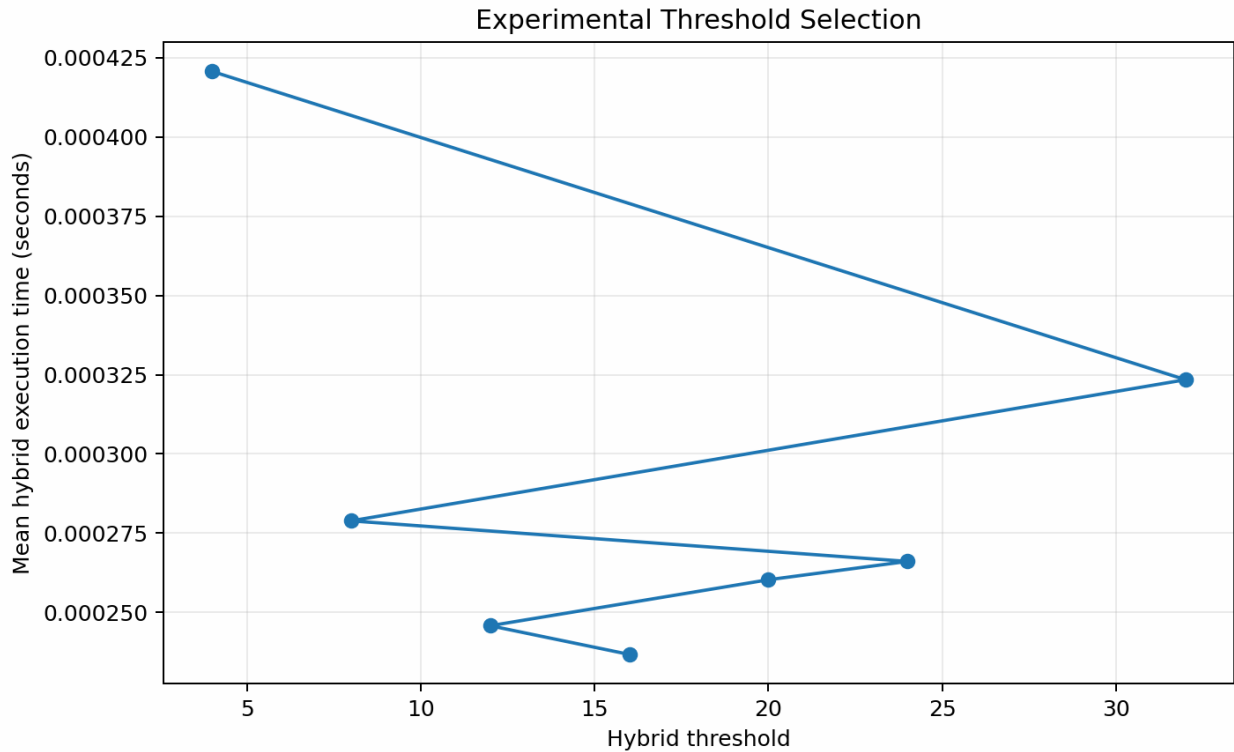


Figure 5. Experimental threshold selection.

n	Brute s	D&C s	Hybrid s	Brute comps	Hybrid comps
10	0.000028	0.000060	0.000027	45	45
20	0.000077	0.000180	0.000075	190	91
40	0.000380	0.000335	0.000162	780	181
60	0.001006	0.000680	0.000293	1770	423
80	0.001739	0.000827	0.000377	3160	364
100	0.002587	0.001040	0.000487	4950	580

Across the six tested input sizes, mean measured times were 0.000970 s for Brute Force, 0.000520 s for Divide and Conquer, and 0.000237 s for Hybrid. The Hybrid mean was about $4.10\times$ faster than Brute Force and $2.20\times$ faster than Divide and Conquer in this small-scale benchmark.

Validation condition: correct = True for every tested size at $T=16$, meaning the Hybrid and Divide-and-Conquer distances matched the Brute-Force result to the implemented numerical tolerance.

6. Analysis and Engineering Decision

- Brute Force has the simplest logic and the lowest algorithmic overhead, so it is appropriate for very small point sets. However, its $\Theta(n^2)$ pair growth makes it unsuitable for large datasets. The 100-point run required 4,950 distance computations.
- Divide and Conquer reduces the candidate comparisons substantially. At $n=100$ it required 142 distance computations in this implementation, versus 4,950 for Brute Force. Its additional ordering and recursion overhead is visible in the memory measurement.
- The Hybrid algorithm combines the advantages of both. At $n=100$ and $T=16$ it required 580 distance computations and had lower measured runtime than either baseline. The Hybrid threshold experiment showed that very small thresholds retain more recursive overhead, while very large thresholds perform too much quadratic work inside the leaves. $T=16$ gave the best measured balance.
- The theoretical and experimental results agree in direction: quadratic growth makes Brute Force increasingly expensive, while Divide and Conquer and Hybrid scale more favourably. Because the working dataset contains only 100 records, implementation benchmark rather than a claim of production-scale performance.
- Engineering decision: the developed Hybrid approach is the most suitable of the three for large-scale geospatial infrastructure data because it retains $\Theta(n \log n)$ asymptotic behaviour while using the lower-overhead Brute-Force solver on small subproblems. The decision is supported by the measured mean runtime and by the reduction in distance computations relative to Brute Force.
- Advantages and limitations. Brute Force: simplest, easy to validate, but poor scalability. Divide and Conquer: strong asymptotic performance, but more implementation complexity and auxiliary memory. Hybrid: combines scalability with lower small-instance overhead, but its threshold is implementation- and hardware-dependent and must be re-tuned when conditions change.

7. Broader Considerations

- Efficient resource utilization is achieved by avoiding unnecessary pairwise distance evaluations. The spatial divide-and-conquer structure reduces the candidate set, while the Hybrid base case avoids recursion where it is not beneficial. This can improve throughput when many infrastructure datasets or repeated planning queries are processed.
- The approach can support infrastructure planning by identifying locations that are unusually close under the selected Euclidean model. A production system could extend the same design to facility screening, service-area analysis, duplicate-location detection, or pre-processing before more detailed spatial analysis.
- Sustainable computational development is supported through reduced computation and better scalability, which can lower processing time and resource demand for repeated analyses.
- SDG 9 – Industry, Innovation and Infrastructure. The solution demonstrates an algorithmic method for making infrastructure-data processing more efficient and scalable. Efficient spatial analytics can contribute to intelligent planning systems by enabling larger datasets to be analyzed with fewer unnecessary computations.

8. Conclusion

- The Closest-Pair-of-Points problem was addressed using a real-world OpenFlights airport-coordinate dataset. Brute Force, Divide and Conquer, and a newly developed Hybrid algorithm were implemented and evaluated under identical conditions.
- The Hybrid algorithm switches to Brute Force for subproblems of size $T=16$ and otherwise uses Divide and Conquer. The threshold was selected experimentally from candidate values 4, 8, 12, 16, 20, 24 and 32.
- For the tested sizes 10–100, all approaches produced the same closest pair. The Hybrid approach had the lowest mean measured execution time in the experiment, approximately 0.000237 s, compared with 0.000970 s for Brute Force and 0.000520 s for Divide and Conquer. The main scalability improvement is the change from $\Theta(n^2)$ pair growth in Brute Force to $\Theta(n \log n)$ asymptotic behaviour for the Divide-and-Conquer and Hybrid methods.
- Limitations include the 100-record working extract, the use of Euclidean rather than geodesic distance, and the dependence of the threshold on implementation and hardware. Future optimization can use the complete OpenFlights dataset, parallel or vectorized distance operations, spatial indexing, geodesic distance when required, and automated threshold tuning.

9. Student Reflection

Answer individually:

1. What did you learn from developing and optimizing the Hybrid Closest-Pair algorithm beyond implementing the algorithms directly taught in the classroom?

The main learning was that algorithm performance is not determined only by Big-O notation. The practical cost of recursion, sorting, memory allocation and function calls matters for small subproblems. I learned how to combine two known strategies into a hybrid design and how to use measured evidence to select a switching point rather than choosing a threshold arbitrarily.

2. What challenges did you encounter while determining the switching threshold and evaluating performance across different input sizes, and how did you address them?

The main challenge was that execution times for small inputs are very short and can be affected by runtime noise. I addressed this by repeating each measurement several times and using the median time. I also compared distance-computation counts, because they provide a more stable algorithmic-work measure than timing alone. Correctness was checked by comparing every Hybrid and Divide-and-Conquer result against Brute Force.

3. If additional computational resources or time were available, how would you further improve the developed algorithm and why?

I would evaluate the complete OpenFlights dataset and much larger subsets, automate threshold tuning over several hardware configurations, and compare Euclidean with geodesic distance. I would also investigate spatial indexing or parallel processing for repeated queries. These changes would improve scalability and make the engineering conclusions more representative of production workloads.

4. How can the developed solution contribute to SDG 9 – Industry, Innovation and Infrastructure?

The solution demonstrates how algorithmic optimization can make infrastructure-data processing faster and more scalable. Reducing unnecessary distance computations can support intelligent infrastructure planning systems that analyze more facilities within the same computational budget.

10. References

- OpenFlights. Airport, airline and route data – Airport Database. <https://openflights.org/data> (accessed September 2, 2026).
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms, 4th ed., MIT Press, 2022.
- Python Software Foundation. Python 3 Documentation – time, csv, math and tracemalloc modules. <https://docs.python.org/3/>
- Matplotlib Development Team. Matplotlib Documentation. <https://matplotlib.org/stable/>
- OpenAI ChatGPT. AI-assisted support was used for organizing the report, drafting implementation structure, and reviewing the experimental methodology; the code was executed and the reported measurements were generated in the supplied execution environment.

F. Assessment Rubric

Criteria	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Algorithmic Formulation	10	Clearly identifies the problem, requirements, inputs, outputs, constraints and computational challenges and formulates them appropriately for solution development.	Problem and major requirements are correctly formulated with minor gaps.	Basic formulation is provided, but requirements or constraints are partially identified.	Problem is poorly understood or inadequately formulated.
Algorithmic Solution Design	15	Designs a clear, logical and feasible solution strategy with appropriate algorithmic techniques, pseudocode/flowchart and well-justified design decisions.	Appropriate solution design with minor gaps in logic, representation or justification.	Basic solution design is presented but lacks completeness or clear justification.	Solution design is inappropriate, incomplete or unclear.
Development / Adaptation / Optimization	20	Develops a meaningful algorithmic solution through integration, adaptation,	Develops an appropriate solution with meaningful	Solution demonstrates limited adaptation or	Merely reproduces an existing algorithm

of Algorithmic Solution		modification, hybridization or optimization of suitable techniques; demonstrates clear improvement or suitability for the identified problem.	adaptation or improvement , with minor limitations.	development beyond standard implementation.	with little or no meaningful development or adaptation.
Implementation & Modern Tool Usage	15	Developed solution is correctly and efficiently implemented using appropriate programming/computational tools and handles relevant input conditions reliably.	Implementation is functional with minor logical or efficiency issues.	Core implementation works but contains limitations or incomplete functionality.	Implementation is incomplete, unreliable or non-functional.
Efficiency, Testing & Validation	15	Performs appropriate complexity evaluation and systematic testing using suitable data/input sizes; validates correctness, efficiency and scalability with clear evidence.	Complexity evaluation and testing are mostly appropriate with minor gaps.	Basic testing and efficiency evaluation are provided but lack sufficient validation.	Testing, complexity evaluation or validation is inadequate or substantially incorrect.
Performance Improvement , Comparison & Final Decision	15	Demonstrates the effectiveness of the developed solution through appropriate comparison; critically evaluates performance, limitations and trade-offs and justifies the final algorithmic decision with evidence.	Good performance comparison and justification with minor limitations.	Basic comparison is provided, but improvement or final justification lacks depth.	Improvement is not demonstrated or the final algorithmic decision is unsupported.
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of algorithmic design and development decisions; clear connection to relevant SDG(s); specific reflection on computational challenges, performance improvements, trade-offs, and learning gained.	Appropriate justification of algorithmic decisions; SDG relevance, challenges, improvements, and learning are discussed but lack depth.	Reflection is present but generic; limited justification of algorithmic decisions, improvements, SDG relevance, or learning outcomes.	No genuine reflection is provided, or the reflection lacks meaningful connection to algorithm development , SDG relevance, challenges, or learning outcomes.
Total	100				

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem Understanding & Algorithmic Formulation	CO1	PO1, PO2	L4	10
Algorithmic Solution Design	CO2, CO3	PO2, PO3	L4, L6	15
Development / Adaptation / Optimization of Algorithmic Solution	CO2, CO3	PO2, PO3	L5, L6	20
Implementation & Modern Tool Usage	CO2, CO3	PO3, PO5	L3, L6	15
Efficiency, Testing & Validation	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Performance Improvement, Comparison & Final Decision	CO1, CO2, CO3	PO2, PO4	L4, L5	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	CO1, CO2, CO3	PO7, PO10, PO12	L4, L5	10

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

